

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA1013	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	S Jefna Princy	Reg. No.:	192572058
Date:	4/08/2026	Duration:	60 minutes

Code of Conduct

I S Jefna Princy- 192572058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

- 1. Study and Analyze the Scenario**
 - Carefully understand the given problem statement.
 - Identify the business domain, stakeholders, existing challenges, and system objectives.
- 2. Identify Functional and Non-Functional Requirements**
 - List all functional requirements required by the proposed system.
 - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
- 3. Identify Actors and Use Cases**
 - Identify all primary and secondary actors interacting with the system.
 - List the major use cases associated with each actor.
 - Draw a Use Case Diagram representing the interactions between actors and the system.
- 4. Prepare the Software Requirement Specification (SRS)**

Develop an SRS document containing:

 - Introduction
 - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. Identify Assumptions and Constraints

- List the assumptions considered while developing the system.
- Identify technical, operational, economic, legal, and time-related constraints affecting the project.

6. Validate Requirement Completeness and Consistency

- Verify whether all stakeholder requirements have been addressed.
- Check for ambiguity, redundancy, inconsistency, and missing requirements.
- Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification	SRS is well-structured, complete, professionally	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

(SRS) Documentation	organized, and includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5

Course code: CSA1013

Name: S Jefna Princy

Course: Software Engineering

Reg No: 192572058

CO 2 ASSESSMENT TOOL - 1

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System

1. Introduction

1.1 Purpose

The purpose of this Software Requirement Specification (SRS) is to describe the requirements for the **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System**. This document explains the features, functions, and performance expected from the system. It acts as a guide for developers, testers, and stakeholders during the design and implementation phases.

The proposed system uses **Artificial Intelligence (AI)**, **Internet of Things (IoT)**, **Geographic Information System (GIS)**, and **Cloud Computing** technologies to continuously monitor urban noise levels. It helps environmental authorities detect noise pollution, identify high-risk locations, generate real-time alerts, and support better urban planning.

1.2 Scope of the System

Noise pollution has become one of the major environmental problems in modern cities due to increasing traffic, industries, construction work, and public activities. Existing manual monitoring methods are slow and cannot provide continuous information about changing noise levels.

The proposed system collects real-time sound data from IoT sensors installed at different locations. The collected data is stored in the cloud and analyzed using AI techniques to identify noise patterns and predict future trends. GIS technology displays noise hotspots on a digital map, while automated alerts notify authorities whenever the permissible noise limit is exceeded.

The system helps environmental agencies, municipal corporations, and urban planners make better decisions to reduce noise pollution and improve the quality of life for citizens.

2. Scenario Understanding and Problem Analysis

2.1 Business Domain

The project belongs to the **Environmental Monitoring and Smart City** domain. Smart cities require intelligent technologies to monitor environmental conditions continuously. Noise pollution affects human health by causing stress, sleep disorders, hearing problems, and reduced productivity.

Government agencies need an automated monitoring system that provides accurate and real-time information about urban noise. The proposed system combines AI, IoT, GIS, and cloud technologies to improve environmental monitoring and support sustainable city development.

2.2 Stakeholders

The following stakeholders interact with the system:

Environmental Protection Agency

- Monitors city noise levels.
- Receives alerts when limits are exceeded.
- Generates environmental reports.

Municipal Corporation

- Uses reports for city planning.
- Identifies high-noise areas.
- Takes corrective actions.

System Administrator

- Manages users and IoT devices.
- Configures system settings.

- Maintains the database and cloud services.

Citizens

- View public noise maps.
- Receive awareness notifications.
- Report excessive noise.

Urban Planners

- Analyze historical noise data.
- Plan quieter residential and commercial areas.

Industries and Construction Companies

- Monitor compliance with environmental noise regulations.
- Take measures to reduce excessive noise.

2.3 Existing Challenges

The current noise monitoring system faces several limitations:

- Noise measurements are collected manually at specific time intervals.
- Continuous monitoring is not possible.
- Authorities cannot identify permanent noise hotspots quickly.
- There is no prediction of future noise levels.
- Reports are prepared manually, which is time-consuming.
- Lack of real-time alerts delays corrective actions.
- Citizens have limited access to environmental information.

These challenges reduce the effectiveness of environmental monitoring and make it difficult to enforce pollution control regulations.

2.4 Objectives

The proposed system aims to achieve the following objectives:

1. Monitor urban noise levels continuously using IoT sensors.
2. Identify noise hotspots through GIS mapping.
3. Predict future noise trends using AI algorithms.
4. Generate real-time alerts when noise exceeds permissible limits.
5. Analyze major sources of noise pollution.
6. Produce environmental compliance reports.
7. Support urban planning and policy-making.
8. Improve public health by reducing exposure to excessive noise.

2.5 Expected Outcomes

After implementing the proposed system, the following outcomes are expected:

- Continuous and accurate noise monitoring.
- Early identification of noise pollution hotspots.
- Faster response from environmental authorities.
- Better compliance with environmental regulations.
- Improved decision-making for urban planning.
- Increased public awareness about noise pollution.
- Reduced environmental noise levels.
- Improved quality of life and public health.

3. Functional Requirements

Functional requirements describe the specific functions and services that the system must perform. These requirements define how the system should respond to user inputs and process data to achieve the project objectives.

3.1 User Authentication

The system shall provide a secure login facility for authorized users such as the System Administrator, Environmental Officer, Municipal Authority, and other registered users. Each user will log in using a unique username and password. The system should verify the credentials before granting access and assign permissions based on the user's role.

3.2 Noise Data Collection

The system shall continuously collect noise level data from IoT-based sound sensors installed at different locations across the city. The sensors will automatically record sound levels in decibels (dB) along with the location and timestamp. The collected data will be transmitted to the cloud server without manual intervention.

3.3 Data Storage and Management

All collected sensor data shall be securely stored in a cloud database. The system should organize and maintain historical noise records for future analysis. The database must support quick retrieval, updating, and backup of information while ensuring data integrity and security.

3.4 AI-Based Noise Analysis

The system shall analyze real-time and historical noise data using Artificial Intelligence and Machine Learning algorithms. AI techniques will identify unusual noise patterns, classify major sources of noise pollution, and estimate future noise levels. This helps authorities take preventive measures before the situation worsens.

3.5 GIS-Based Noise Hotspot Detection

The system shall use Geographic Information System (GIS) technology to display the collected noise data on a digital map. Areas where noise levels frequently exceed permissible limits will be marked as **noise hotspots**. Different colors can be used to indicate low, moderate, and high noise intensity, making it easier for authorities to identify affected regions.

3.6 Real-Time Alert Generation

Whenever the measured noise level exceeds the permissible limit set by environmental regulations, the system shall automatically generate alerts. Notifications may be sent through SMS, email, or dashboard messages to environmental officers and municipal authorities, enabling them to respond immediately.

3.7 Dashboard and Report Generation

The system shall provide an interactive dashboard displaying live noise levels, hotspot locations, AI predictions, sensor status, and environmental statistics. Users should also be able to generate daily, weekly, monthly, and annual reports. These reports can be downloaded in formats such as PDF or Excel for official use.

3.8 User and Sensor Management

The System Administrator shall have the authority to add, update, or remove users and IoT sensors. The administrator can also configure system settings, manage user roles, monitor sensor health, and maintain the overall performance of the platform.

4. Non-Functional Requirements

Non-functional requirements specify the quality attributes and performance standards that the system must satisfy to ensure efficient and reliable operation.

4.1 Performance

The system should process incoming sensor data within a few seconds and update the dashboard in real time. It should be capable of handling data from multiple IoT sensors simultaneously without affecting system performance.

4.2 Security

The system shall ensure secure user authentication through usernames and passwords. Role-based access control should be implemented so that users can access only the features relevant to their responsibilities. Sensitive data stored in the cloud should be protected using encryption techniques.

4.3 Reliability

The system should operate continuously with minimal downtime. Automatic backup and recovery mechanisms should be available to prevent data loss in case of hardware or software failures.

4.4 Availability

The platform should be available 24 hours a day and 7 days a week to support continuous environmental monitoring. Scheduled maintenance should not significantly interrupt system operations.

4.5 Scalability

The system should support the addition of new IoT sensors, users, and monitoring locations without requiring major changes to the software. It should also be capable of expanding to monitor multiple cities in the future.

4.6 Usability

The user interface should be simple, interactive, and easy to navigate. Dashboards, reports, and GIS maps should be presented in a clear format so that users with basic technical knowledge can operate the system efficiently.

4.7 Maintainability

The software should follow a modular design so that new features, updates, and bug fixes can be implemented easily without affecting existing functionalities.

4.8 Accuracy

The AI prediction model should provide accurate noise analysis and forecasting based on historical and real-time data. The system should minimize false alerts and ensure reliable environmental monitoring.

5. Actors and Use Cases

A **use case** describes how different users (actors) interact with the proposed system to perform various operations. It helps in understanding the system's functionality from the user's perspective.

5.1 Actors

1. System Administrator

The System Administrator is responsible for managing the overall system. The administrator registers users, manages IoT sensors, configures system settings, monitors system performance, and ensures that the platform functions smoothly.

2. Environmental Officer

The Environmental Officer continuously monitors urban noise levels through the dashboard. The officer receives alerts whenever noise exceeds the permissible limit, analyzes reports, and initiates corrective actions to reduce noise pollution.

3. Municipal Authority

The Municipal Authority uses the system to study noise pollution trends, identify high-risk zones, and make decisions regarding urban planning and environmental regulations. They also review compliance reports generated by the system.

4. Citizens

Citizens can access the public dashboard to view noise pollution levels in different areas of the city. They receive awareness notifications and may report excessive noise to the concerned authorities.

5. IoT Sensors

IoT sound sensors automatically collect environmental noise data from different urban locations and send the collected information to the cloud server for further analysis.

6. AI Prediction Module

ACTORS AND THEIR ROLES		
Actor	Role Description	LEGEND
 System Administrator	Manages users, system settings, sensors and overall system.	 Use Case  Actor  Association «include» Include «extend» Extend
 Environmental Officer	Monitors noise levels, detects hotspots and generates alerts.	
 Government Official	Views reports, analytics and takes corrective actions.	
 Urban Planner	Uses data for urban planning and policy making.	
 Citizen	Views public dashboard and awareness information.	
 IoT Sensor (External System)	Collects and sends noise data to the system.	

The
AI

module analyzes historical and real-time data to identify noise patterns, predict future noise levels, and detect abnormal environmental conditions.

5.2 Use Cases

The following table shows the major use cases performed by each actor.

5.3 Use Case Descriptions

Use Case 1 – User Login

Actor: System Administrator, Environmental Officer, Municipal Authority

Description:

The user enters a valid username and password to access the system. After successful authentication, the system displays the dashboard according to the user's role.

Pre-condition:

The user must be registered in the system.

Post-condition:

The user gains access to the system dashboard.

Use Case 2 – Collect Noise Data

Actor: IoT Sensor

Description:

The IoT sensor continuously measures environmental noise levels and transmits the collected data to the cloud server. The data includes noise level, location, and timestamp.

Pre-condition:

Sensor must be connected to the network.

Post-condition:

Noise data is successfully stored in the database.

Use Case 3 – Detect Noise Hotspots

Actor: AI Prediction Module

Description:

The AI module analyzes the collected data and identifies areas where the recorded noise

levels frequently exceed the permissible limit. These locations are highlighted as noise hotspots on the GIS map.

Pre-condition:

Sufficient historical and real-time data must be available.

Post-condition:

Hotspots are displayed on the GIS dashboard.

Use Case 4 – Generate Environmental Alerts

Actor: Environmental Officer

Description:

Whenever the system detects excessive noise levels, it automatically generates alerts and notifies the concerned authorities through the dashboard, email, or SMS.

Pre-condition:

Noise level exceeds the prescribed limit.

Post-condition:

Alert is delivered to the responsible authority.

Use Case 5 – Generate Reports

Actor: Environmental Officer, Municipal Authority

Description:

The system generates daily, weekly, monthly, or annual reports containing noise statistics, hotspot analysis, AI predictions, and compliance status. These reports assist authorities in monitoring environmental conditions and planning corrective measures.

Pre-condition:

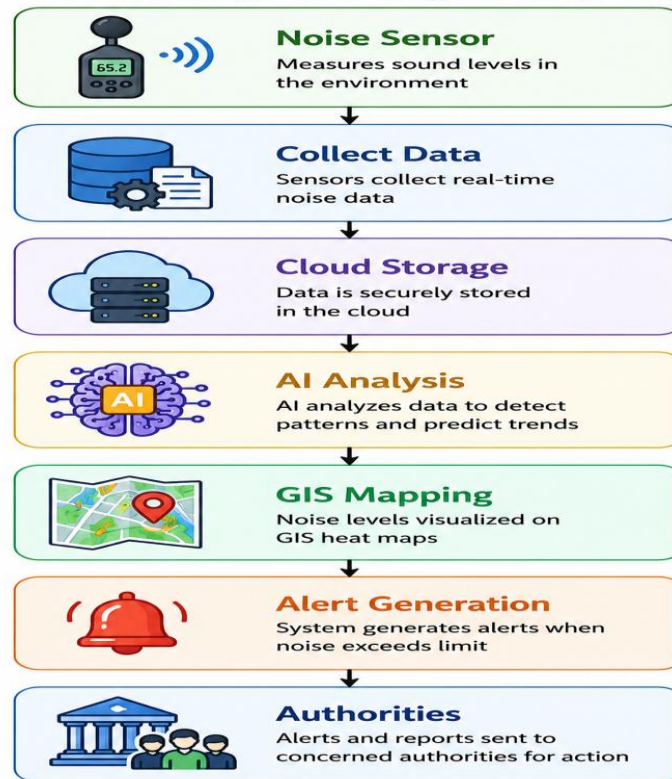
Noise data must be available in the database.

Post-condition:

Report is generated and can be downloaded in PDF or Excel format.

5.4 Use Case Diagram

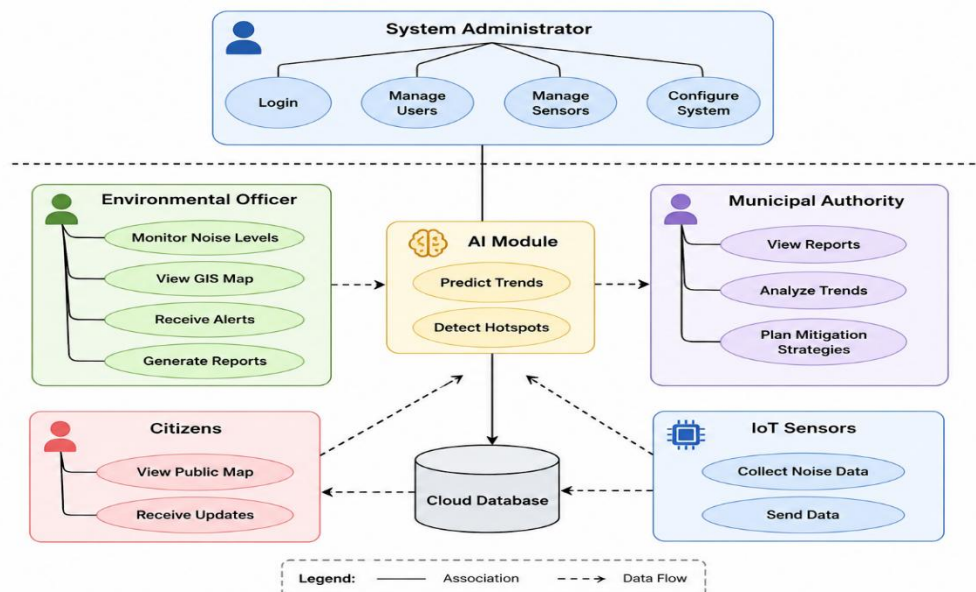
AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System



6.

Software Requirement Specification (SRS)

6.1



Problem Description

Urban noise pollution has become a serious environmental issue due to rapid urbanization, increasing traffic, industrial activities, construction work, and public events. Excessive noise affects human health by causing hearing loss, stress, sleep disturbances, reduced concentration, and cardiovascular problems. Traditional noise monitoring methods rely on manual measurements taken at specific intervals, making it difficult to monitor continuous changes in noise levels. As a result, authorities often fail to identify persistent noise hotspots and cannot take timely corrective actions.

The proposed **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** addresses these limitations by integrating IoT sensors, Artificial Intelligence (AI), Geographic Information System (GIS), and cloud computing technologies. The system continuously collects noise data from multiple locations, analyzes the data using AI algorithms, identifies high-risk zones, predicts future noise trends, and generates real-time alerts whenever permissible limits are exceeded. This enables authorities to make informed decisions and implement effective noise control measures.

6.2 Scope of the System

The proposed system is designed to monitor and analyze urban noise pollution in real time. It provides continuous monitoring through IoT sensors installed across different parts of the city. The collected data is transmitted to a cloud server, where AI models analyze the information to detect abnormal noise patterns and predict future trends.

The system also uses GIS technology to display noise hotspots on an interactive digital map. Environmental officers and municipal authorities receive alerts when excessive noise is detected and can generate reports for regulatory compliance. The platform supports data-driven decision-making for urban planning, environmental protection, and public health improvement.

Future enhancements may include mobile application support, citizen feedback, integration with weather monitoring systems, and expansion to multiple cities.

6.3 System Features

The proposed system includes the following major features:

- **Real-Time Noise Monitoring**

The system continuously collects environmental noise data from IoT sound sensors installed at various locations. Noise levels are recorded in decibels (dB) and updated in real time.

- **AI-Based Noise Analysis**

Artificial Intelligence analyzes both historical and live sensor data to identify unusual noise patterns, classify noise sources, and predict future noise levels.

- **GIS Hotspot Mapping**

Noise levels are displayed on an interactive GIS map. Locations with consistently high noise levels are highlighted as hotspots using different color indicators, making it easier for authorities to identify affected areas.

- **Automated Alert System**

Whenever the recorded noise exceeds the permissible environmental limit, the system automatically generates alerts and sends notifications to the concerned authorities through the dashboard, email, or SMS.

- **Dashboard and Visualization**

The system provides a user-friendly dashboard displaying live noise readings, hotspot locations, AI predictions, sensor status, and statistical graphs for better understanding of environmental conditions.

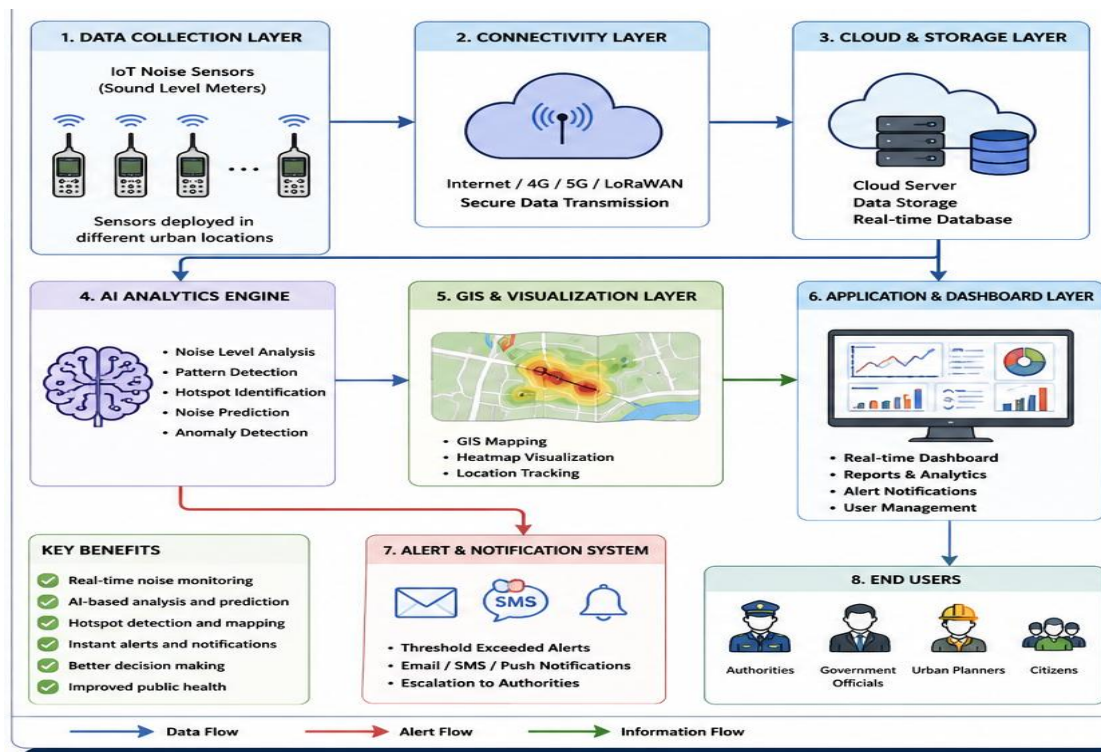
- **Report Generation**

Users can generate daily, weekly, monthly, and annual reports containing noise statistics, hotspot analysis, AI predictions, and compliance information. Reports can be downloaded in PDF or Excel format.

- **User and Sensor Management**

The administrator can manage user accounts, configure system settings, register IoT sensors, and monitor sensor status to ensure smooth system operation.

6.4 Data Requirements



The system requires various types of input and output data for effective monitoring and analysis.

Input Data

The following information is collected by the system:

- Sensor ID
- Location Name
- GPS Coordinates
- Date and Time
- Noise Level (dB)
- Temperature (Optional)
- Weather Conditions (Optional)

Output Data

The system produces the following outputs:

- Real-time noise dashboard
- GIS-based hotspot map
- Noise prediction results
- Environmental alerts
- Daily, weekly, monthly, and annual reports
- Compliance reports
- Statistical graphs and charts

Database Requirements

The cloud database stores:

- User details
- Sensor information
- Historical noise records
- Alert history
- AI prediction results
- Report history
- GIS location data

The database should support secure storage, quick retrieval, automatic backup, and data recovery.

6.5 External Interface Requirements

External interfaces define how the system interacts with users, hardware devices, and other software applications.

Hardware Interface

The system requires the following hardware components:

- IoT Sound Sensors
- GPS Module

- Cloud Server
- Desktop/Laptop
- Mobile Devices
- Internet Connectivity

The sensors collect environmental noise data and transmit it to the cloud server for processing.

Software Interface

The proposed system interacts with various software components, including:

- Cloud Computing Platform
- AI/Machine Learning Models
- GIS Mapping Software
- Database Management System (MySQL or PostgreSQL)
- Web Browser

These software components work together to collect, analyze, store, and visualize environmental data.

User Interface

The system provides a simple and interactive graphical user interface (GUI) that allows users to:

- Securely log in to the system.
- View live noise levels.
- Access GIS-based hotspot maps.
- Receive real-time alerts.
- Generate and download reports.
- Manage users and IoT sensors.
- Analyze AI prediction results.

The interface should be user-friendly so that even non-technical users can easily operate the system.

7. Assumptions and Constraints

7.1 Assumptions

Assumptions are the conditions that are expected to be true for the successful development and operation of the system.

- IoT noise sensors are properly installed and calibrated at different urban locations.
- A stable internet connection is available for transmitting sensor data to the cloud server.
- The cloud platform has sufficient storage and processing capacity to handle large volumes of sensor data.
- AI models are trained using accurate and sufficient historical noise data to ensure reliable predictions.
- Users such as environmental officers, municipal authorities, and administrators have the necessary training to operate the system.
- GIS services and digital maps are available for displaying noise hotspot locations.
- Environmental noise limits are defined according to government regulations and remain available for comparison.
- Regular maintenance of sensors and system components is carried out to ensure continuous monitoring.

7.2 Constraints

The development and operation of the proposed system may be affected by the following constraints:

Technical Constraints

- The accuracy of monitoring depends on the quality and calibration of IoT sensors.
- Poor internet connectivity may delay real-time data transmission.
- AI prediction accuracy depends on the availability of sufficient historical data.

- System performance may be affected if a large number of sensors transmit data simultaneously.

Operational Constraints

- Regular maintenance and calibration of sensors are required.
- Trained personnel are needed to monitor reports and respond to alerts.
- Sensor failures may temporarily interrupt data collection in specific locations.

Economic Constraints

- Initial installation of IoT sensors and cloud infrastructure involves significant cost.
- Ongoing expenses include sensor maintenance, cloud storage, software updates, and technical support.

Legal Constraints

- The system must comply with national environmental noise regulations.
- User and environmental data must be protected according to data privacy and security policies.
- Reports generated by the system should meet environmental compliance standards.

Time Constraints

- Noise data should be processed within a few seconds to support real-time monitoring.
- Environmental alerts should be generated immediately when noise levels exceed the permissible limit.
- Periodic reports should be generated automatically based on predefined schedules.

8. Requirement Validation

Requirement validation ensures that the proposed system requirements are complete, consistent, feasible, and meet stakeholder expectations.

8.1 Requirement Completeness

The requirements have been carefully analyzed to ensure that all major functionalities expected from the system are included.

The SRS covers:

- Introduction
- Problem Description
- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Actors and Use Cases
- Data Requirements
- External Interface Requirements
- System Features
- Assumptions and Constraints

This ensures that the proposed system satisfies the needs of all stakeholders.

8.2 Requirement Consistency

The identified requirements are consistent with the objectives of the project.

- Functional and non-functional requirements do not conflict with each other.
- Every system feature supports at least one project objective.
- User roles and responsibilities are clearly defined.
- The requirements are realistic and technically achievable using AI, IoT, GIS, and cloud technologies.

8.3 Requirement Feasibility

The proposed system is technically and economically feasible because:

- IoT sensors are widely available for environmental monitoring.
- AI and machine learning techniques can accurately analyze and predict noise patterns.
- GIS technology provides effective visualization of noise hotspots.

- Cloud computing enables secure storage and real-time processing of environmental data.

8.4 Suggested Improvements

To further enhance the proposed system, the following features may be included in future versions:

- Develop a mobile application for citizens and environmental officers.
- Integrate weather information to improve AI prediction accuracy.
- Support multilingual user interfaces for wider accessibility.
- Enable citizen complaint registration through the mobile application.
- Implement advanced deep learning models for more accurate noise prediction.
- Integrate smart traffic management systems to reduce traffic-related noise pollution.